Docking demonstrator: ROS 2 + Python implementation brief

Standalone guide for porting the docking estimator and controller to the tabletop demonstrator (Raspberry Pi 5, BNO085, camera with a fiducial marker on the target, four VEX mecanum wheels with speed-controlled motors). It assumes the design is fixed: the frames, states, equations and parameters are those of `docking_formulation.pdf`, `ekf_equations.pdf` and `mpc_bridge.pdf`, and the MATLAB in `matlab/` is the executable specification. Everything below is about turning that into running software, calibrating it, and proving it on the bench.

Conventions used throughout: body velocity `u = [u_b, v_b, r_b]` is forward, left, counter-clockwise; wheel order is FL, FR, RL, RR; angles in radians, lengths in metres, inside the code; the target frame `T` has its origin where the chassis centre sits when docked and `x_T` points into the target, so the robot approaches from `x < 0` with `v_x > 0`.

1. Hardware and software baseline

Item	Choice	Notes
Computer	Raspberry Pi 5, 8 GB, active cooling	Fix the CPU governor to `performance`; the MPC solve time doubles when the clock drops.
OS / ROS	Ubuntu 24.04 + ROS 2 Jazzy (or 22.04 + Humble)	Python 3.12 / 3.10. Build in a workspace `~/dock_ws`.
Python packages	`numpy`, `scipy`, `osqp`, `opencv-python` (contrib for ArUco) or `pupil-apriltags`, `pyserial`	`osqp` is the QP solver on the Pi; `quadprog` was the MATLAB stand-in.
IMU	BNO085 over I2C at 400 kHz (or UART-RVC)	Use the *linear acceleration* report (gravity removed by the onboard fusion) and the calibrated gyro. Request 200 Hz for both. The `adafruit-bno08x` driver works; a custom SH-2 reader is faster.
Camera	Global-shutter module preferred (Pi GS camera or an Arducam OV9281); rolling shutter acceptable at the speeds involved	640 x 480 at 30 fps, processed at 15 fps. Mount rigidly; measure the lever arm `l_c` from the chassis centre.
Marker	One AprilTag (36h11) or ArUco (4x4_50) of known size on the target, at a known offset `(m, psi_0)` from the docked origin	Larger is better for yaw accuracy; 80 to 120 mm for the tabletop.
Motors and drive	Four VEX motors with the custom speed controller	Interface contract in Section 8. The design assumes wheel-speed loops closed at >= 100 Hz below the Pi.
Safety	Hardware E-stop cutting motor power; software watchdog in the wheel node	Both required before the first closed-loop run.

Set the Pi up for soft real time: isolate one core for the EKF and one for the MPC (`isolcpus` or `taskset`), run the nodes with elevated priority (`chrt -f 50`), and turn off wifi power saving. None of the loops need hard real time, but a 50 ms MPC period with a 25 ms solve budget does not tolerate 100 ms scheduling gaps.

2. Frames, transforms and the fiducial chain

TF tree (static unless stated):

```
target  --(static: marker offset m, psi_0)-->  marker
base_link --(static: camera mount)-->  camera_link
base_link --(static: IMU mount)-->  imu_link
target  --(dynamic, published by the EKF node)-->  base_link
```

Frame definitions:

- `target`: origin at the docked chassis-centre position, `x` into the target, `y` left, `z` up. Reference pose for docking is `(0, 0, 0)`.
- `marker`: the fiducial, at position `m` (2-D) and yaw `psi_0` in `target`, both measured once from the target's geometry.
- `base_link`: chassis centre, `x` forward, `y` left. The coupler is at `(L_k, 0)` in `base_link`.

- `camera_link`: optical centre, with the usual optical-frame convention (`z` forward, `x` right, `y` down) handled by a fixed rotation inside the fiducial node.

The fiducial node turns a raw solver output into the pseudo-measurement the EKF expects. The solver returns the marker pose in the camera frame as a rotation `R_CM` and translation `t_CM`. The chain is

```
T_TB = T_TM * inv(T_CM) * inv(T_BC)
```

with `T_TM` the static marker offset and `T_BC` the static camera mount. Take `x`, `y` and yaw out of `T_TB`; that is `z = [p_z; theta_z]`. The planar form in the equation sheet, `theta_z = psi_0 - psi` and `p_z = m - R(theta_z) (t + l_c)`, is this chain with the camera aligned to the body; use the full chain in code and keep the planar form for the report. Roll and pitch of `T_TB` are the monitored channels (not controlled); publish them for the supervisor.

Sign checks that must pass before anything else runs (Section 10.b): place the robot at four known poses (straight in front, offset left, offset right, yawed 15 degrees) and compare `z` with a tape measure. Every sign error found later cost a day in simulation; on hardware it costs a wheel.

---

3. Node graph

All nodes in one Python package `dock_control`, one launch file. Topics carry the capture time of the data they describe, never the publish time.

Node	Rate	Subscribes	Publishes	Role
`imu_node`	200 Hz	hardware	`/imu/data` (`sensor_msgs/Imu`)	Linear acceleration (gravity removed) in `imu_link`, rotated into `base_link`; gyro `z`. Stamp from the sensor's own clock, offset-corrected once at start.
`fiducial_node`	15 Hz	camera	`/fiducial/fix` (`FiducialFix`)	Detect, solve PnP, apply the chain of Section 2, publish `z`, `sigma_p(d)`, `sigma_th`, `valid`, `marker_id`, `capture_stamp`.
`ekf_node`	200 Hz	`/imu/data`, `/fiducial/fix`	`/dock/estimate` (`DockEstimate`) at 200 Hz, `tf target->base_link` at 20 Hz	Predict on every IMU sample; correct on every fix against the ring-buffered state at the fix's capture stamp; gate; publish state, covariance diagonal, gyro, fix age, NIS of the last fix.
`supervisor_node`	20 Hz	`/dock/estimate`, `/fiducial/fix`, `/estop`	`/dock/stage` (`DockStage`)	State machine of Section 7: reference pose, speed cap, flags.
`mpc_node`	20 Hz	`/dock/estimate`, `/dock/stage`	`/dock/cmd_body` (`geometry_msgs/TwistStamped`), `/dock/mpc_info` (`MpcInfo`)	One QP per tick; command is the body velocity; info carries the predicted trajectory, slack, exit flag and solve time.
`wheel_node`	50 Hz (on command)	`/dock/cmd_body`, motor feedback	motor set-points, `/wheels/state` (`JointState`)	Inverse kinematics, saturation as a last resort, 100 ms command watchdog, zero on E-stop.
`bag`	—	everything	—	`ros2 bag record -a` on every run; the thesis data comes from bags.

Custom messages (package `dock_msgs`):

```
FiducialFix.msg      std_msgs/Header header (stamp = capture time)
                     float64 x, y, theta         # z in the target frame
                     float64 sigma_p, sigma_th   # per-fix standard deviations
                     float64 roll, pitch         # monitored only
                     float64 range               # |t_CM|, for the noise model and logging
                     int32 marker_id
                     bool valid
DockEstimate.msg     std_msgs/Header header (stamp = IMU sample time)
                     float64[5] x                # [x, y, theta, vx, vy] in target
```

```
                    float64[5] p_diag
                    float64 r_gyro
                    float64 fix_age             # seconds since the last accepted fix
                    float64 last_nis            # nu' S^-1 nu of the last fix
                    bool last_fix_accepted
DockStage.msg       uint8 stage                 # IDLE, ACQUIRE, APPROACH, FINAL, DOCKED,
HOLD, ABORT
                    float64[3] y_ref            # reference pose
                    float64 v_max, r_max        # caps for this tick
                    bool command_enabled
MpcInfo.msg         std_msgs/Header header
                    float64[] y_pred            # 3*Np predicted outputs
                    float64 slack
                    int32 exit_flag
                    float64 solve_ms
                    bool fallback
```

Executor layout: `imu_node` and `ekf_node` in one process on one core (the EKF callback must run within 1 ms of each IMU sample); `mpc_node` and `supervisor_node` in a second process; `fiducial_node` in its own process (the solver is the CPU hog). Use `rclpy` `MultiThreadedExecutor` only where callbacks must overlap; the EKF must not.

---

4. Timing and latency budget

Path	Budget	How it is met
IMU sample to EKF predict done	< 1 ms	Vectorised 5-state predict is about 50 us in NumPy; the cost is the I2C read. Read in a tight loop, not from a ROS timer.
Camera exposure to fix published	40 to 80 ms	Detection 10 to 30 ms, PnP 1 ms; stamp the frame at exposure (libcamera gives it), not at receipt. The EKF uses the stamp; the latency itself only has to be stable.
Fix arrival to correction applied	< 2 ms	Ring buffer lookup by stamp, one 3x3 solve.
MPC tick: latest estimate to command published	< 25 ms	OSQP on a 31-variable, 320-row sparse problem is 3 to 8 ms on the Pi 5 with warm start; the matrix build in NumPy is 2 to 4 ms.
Command to wheel set-point sent	< 2 ms	Wheel node is a pass-through.

The MPC runs on a 50 ms timer and takes the most recent estimate at the moment it fires; it does not wait for a particular IMU sample. The command is held by the wheel node until the next one. A tick that misses its budget is logged (`solve_ms` in `MpcInfo`) and the previous command stays in force; two consecutive misses trigger the supervisor's HOLD.

Clocks: all stamps in ROS time on one machine, so no synchronisation problem. The IMU's internal timestamp drifts; re-anchor it to ROS time once per second with a first-order filter rather than trusting either alone.

---

5. The EKF node

State `x = [x, y, theta, vx, vy]` in `target`, inputs `[a_x, a_y, r]` from the IMU. The equations are steps 1 to 11 of `ekf_equations.pdf`; the MATLAB in `matlab/ekf/` is the line-by-line reference.

Predict, on every IMU sample (`T = 1/200`):

```
def predict(self, a, r):
    th = self.x[2]; c, s = cos(th), sin(th)
    F = eye(5); F[0,3] = T; F[1,4] = T
    F[3,2] = -T*(s*a[0] + c*a[1]); F[4,2] = T*(c*a[0] - s*a[1])
    G = zeros((5,3)); G[2,2] = T
    G[3,0], G[3,1] = T*c, -T*s
    G[4,0], G[4,1] = T*s,  T*c
    self.x += T*array([self.x[3], self.x[4], r, c*a[0]-s*a[1], s*a[0]+c*a[1]])
    self.x[2] = wrap_pi(self.x[2])
    self.P = F @ self.P @ F.T + G @ self.Sigma_u @ G.T + self.Q_floor
    self.P = 0.5*(self.P + self.P.T)
```

```
    self.buffer.push(stamp, self.x.copy())
```

Update, on every fix, against the buffered state at the capture stamp:

```
def update(self, z, R, stamp):
    x_ref = self.buffer.at(stamp)            # nearest sample at or before the stamp
    nu = z - H @ x_ref; nu[2] = wrap_pi(nu[2])
    S = H @ self.P @ H.T + R
    nis = nu @ solve(S, nu)
    if nis > self.gate:                       # 100: gross errors only
        return False, nis
    K = solve(S.T, (self.P @ H.T).T).T        # P H' S^-1 without inv
    self.x += K @ nu; self.x[2] = wrap_pi(self.x[2])
    IKH = eye(5) - K @ H
    self.P = IKH @ self.P @ IKH.T + K @ R @ K.T
    self.P = 0.5*(self.P + self.P.T)
    self.buffer.overwrite_from(stamp, self.x)  # optional; see below
    return True, nis
```

Details that matter on hardware:

- **Ring buffer.** Depth 0.5 s (100 samples), keyed by stamp. `at(stamp)` returns the latest entry with `t <= stamp`. The correction is applied to the current state; the buffer entries after the capture stamp are not re-propagated (the design accepts the second-order error; the simulation shows 1.6 mm residual against 10 mm uncompensated).
- **Initialisation.** First valid fix: `x0 = [z; 0; 0]`, `P0 = blkdiag(R_fix, sigma_v0^2 I_2)`. Until then the node publishes nothing and the supervisor stays in ACQUIRE.
- **IMU zeroing.** The design treats residual accelerometer bias as noise, not as a state. Before every session, with the robot stationary, average 2 s of linear acceleration and subtract it; log the value. The gravity leak from a 0.2 degree attitude error is 0.03 m/s^2, which is the `bias_a` the simulation assumes; if the bench shows more, raise `q_v`.
- **R per fix.** `R = diag(sigma_p(d)^2, sigma_p(d)^2, sigma_th^2)` with `sigma_p(d) = sigma_p0 * sqrt(1 + (d/d_0)^2)` and the constants from the bench table of Section 10.b, not from the simulation defaults.
- **Dropout.** No fix for `t_hold = 0.5 s` is reported in `fix_age`; the supervisor decides. The EKF never coasts on its own policy.
- **Heading wrap** in three places: after predict, in the innovation, after the correction. Missing any one produces a 358 degree correction at some point.
- **Logging.** Publish `last_nis` on every fix. The histogram of NIS over a session is the consistency check on hardware (mean near 3 for a 3-dof fix) and the evidence for the gate value.

Parameters (YAML): `T`, `Sigma_u` (as `q_scale * diag(sigma_a^2, sigma_a^2, sigma_r^2)`), `Q_floor` (`[1e-8, 1e-8, q_th, q_v, q_v]`), `sigma_v0`, `gate`, the `sigma_p0, d_0, sigma_th` of the fiducial, buffer depth. Mirror `dock_params('demo')` exactly and keep one file.

---

6. The MPC node

One tick, in order (`mpc_bridge.pdf` Steps 1 to 8; MATLAB `matlab/mpc/mpc_step.m` is the reference):

1. Read the latest estimate and the stage. Build `x_p = [T_s*[vx, vy, r_gyro]; [x, y, theta]]`.
2. Linearise at `(x_hat_c, u_prev)`: `A_k`, `B_k` (3x3 each, `mpc_linearise`).
3. Build `F` (120x6) and `Phi` (120x30) for `N_p = 40`, `N_c = 10` (`mpc_predict_matrices`). Build with the recursion, not matrix powers.
4. Cost: `E = 2 (Phi' Qbar Phi + Rbar)`, `f = -2 Phi' Qbar (Y_r - F x_p)`, with `Qbar = I kron Q_y`, `Rbar = r_w I kron R_u`.
5. Constraints `M dU <= gamma` (`mpc_constraints`): increments (60 rows), L1 speed cap and yaw-rate cap (60 rows), wheel speeds through `J^-` (80 rows), no-penetration (40 rows), heading corridor (80 rows). The first 200 rows have constant left-hand sides; precompute them once. The last 120 depend on `Phi` and change every tick.
6. Extend with the slack: `z = [dU; s]`, `E_ext = blkdiag(E, 2e2)`, `f_ext = [f; rho_s]`, soft rows get a `-1` in the slack column, `s >= 0`.

7. Solve. Apply `du = z[0:3]`, `u = u_prev + du`. Publish `u`; publish `Y = F x_p + Phi dU` reshaped to `N_p x 3` in `MpcInfo`.
8. Fallback if the solver fails: decelerate each axis by its largest increment towards zero and set `fallback = true`.

OSQP mapping. OSQP solves `min 1/2 z'Pz + q'z` subject to `l <= A z <= u`. Use

```
P = E_ext (sparse CSC), q = f_ext
A = [M_ext; e_s'] where e_s' selects the slack
l = [-inf * ones(320); 0], u = [gamma; +inf]
```

Settings: `eps_abs = eps_rel = 1e-4`, `max_iter = 4000`, `polish = True`, `warm_start = True`, `verbose = False`. Keep one `osqp.OSQP()` object alive and call `update(q=..., l=..., u=..., Ax=...)` for the parts that change (the sparsity pattern of `A` is constant because the output blocks are dense in the same positions every tick; only the values change). Warm-start `z` from the previous solution shifted by one block. Check `res.info.status_val == 1` (solved) or `2` (solved inaccurate, acceptable); anything else is the fallback. OSQP handles the degenerate vertex that stalled MATLAB's active-set, so the two-solver retry is not needed.

Expected numbers on the Pi 5: matrix build 2 to 4 ms, solve 3 to 8 ms, total under 15 ms against the 50 ms period. If it is slower, the first thing to check is that `numpy` is using the system BLAS and the CPU governor is on `performance`.

The output to the wheel node is the body velocity `u`; the wheel node applies `J^-`. The MPC already respects the wheel limits through its constraints, so the wheel node's saturation should never engage; count the times it does and treat any nonzero count as a bug.

Parameters (YAML): `T_s, N_p, N_c, Q_y, R_u, r_w, du_max, v_max_approach, v_max_final, r_max, w_max, x_max (contact band), th_max, rho_s`, geometry `R_w, l_x, l_y`. Same file as the EKF.

---

7. The supervisor

A state machine at 20 Hz. Thresholds are the ones in `dock_params`: standoff 0.30 m, `align_tol = (10 mm, 1 degree)`, `standoff_tol = 50 mm`, `dock_tol = 3 mm`, `dock_vtol = 20 mm/s`.

State	Reference	Cap	Enter when	Leave when
IDLE	—	0	power on	operator arms
ACQUIRE	—	0	armed	first valid fix accepted (EKF initialised) -> APPROACH
APPROACH	standoff `(-0.30, 0, 0)`	`v_approach = 0.30`	from ACQUIRE	`|y| < 10 mm`, `|theta| < 1 deg`, `|x + 0.30| < 50 mm` -> FINAL
FINAL	origin `(0, 0, 0)`	lowered at the increment rate to `v_final = 0.05`	from APPROACH	`x >= -3 mm`, speed `< 20 mm/s`, `|y| <= 4 mm`, `|theta| <= 1.5 deg` (all from the estimate) -> DOCKED
DOCKED	—	0	from FINAL	operator
HOLD	hold the current pose	`v_final`	`fix_age > 0.5 s` in APPROACH or FINAL, or MPC fallback twice in a row	fix returns within 2 s -> previous state; else ABORT
ABORT	—	0	from HOLD timeout, E-stop, estimate covariance above 0.05 m, or any pose outside the corridor	operator

The FINAL cap must come down at the increment rate: `v_max(k) = max(v_final, |u_b| + |v_b| - 2 * du_max)` from the last command, never stepped, or the amplitude rows become infeasible. The supervisor publishes `v_max` every tick and the MPC uses what it is given.

The DOCKED declaration uses the estimate, which is a few millimetres uncertain; the coupler's engagement band absorbs that. Log the true final pose from an external measurement for the thesis (Section 11).

---

8. The wheel node and the motor controller contract

The design assumes a motor controller that

- accepts four wheel-speed set-points in rad/s at 50 Hz or faster, over serial or CAN,
- closes a speed loop per wheel at >= 100 Hz with its own encoder,
- reports measured wheel speeds at >= 50 Hz,
- stops on its own if no set-point arrives for 100 ms.

If the custom controller does not close the speed loop, the wheel node must: read encoders at 200 Hz, run four PI loops, and output PWM. In that case the loop time constant `tau` is whatever those PI loops give, and it must be measured (Section 10.c) because the kinematic MPC tolerates a lag of 0.1 s and starts overshooting the band beyond about 0.3 s.

The wheel node itself:

```
omega = J_inv @ u            # J_inv = (1/R_w) [[1,-1,-L],[1,1,L],[1,1,-L],[1,-1,L]], L = l_x +
l_y
scale = max(1, max(abs(omega)) / w_max)
omega /= scale               # should never engage; count it
send(omega); watchdog.reset()
```

Zero the set-points immediately on E-stop, on command timeout, and on the supervisor's `command_enabled = false`.

9. Parameter file

One YAML, loaded by every node, mirroring `dock_params('demo')` exactly. Values that the bench will replace are marked.

```
dock:
  imu:   {rate: 200, sigma_a: 0.02, sigma_r_deg: 0.15, q_scale: 4}       # bench
  fid:   {rate: 15, delay_nominal: 0.05, sigma_p0: 0.004, sigma_th_deg: 0.5, d0: 1.0, gate:
100}   # bench
  ekf:   {q_v: 4.5e-6, q_th: 3.8e-9, p_floor: 1.0e-8, sigma_v0: 0.1, buffer_s: 0.5}
  geom:  {Rw: 0.0508, lx: 0.15, ly: 0.15, Lk: 0.15, cam_xyz: [0.10, 0.0, 0.08], marker_m:
[0.30, 0.0], marker_psi0_deg: 180}   # measure
  lim:   {v_approach: 0.30, v_final: 0.05, a_max: 0.5, r_max: 0.5, alpha_max: 1.0, w_max: 15.0}
  # w_max: measure
  mpc:   {Ts: 0.05, Np: 40, Nc: 10, rw: 1.0, contact_band: 0.010, th_max_deg: 20, Qy: [6944,
6944, 1459], Ru: [1600, 1600, 400], rho_s: 5.8e6}
  sup:   {standoff: 0.30, align_tol_m: 0.010, align_tol_deg: 1.0, standoff_tol: 0.05, dock_tol:
0.003, dock_vtol: 0.02, hold_after_s: 0.5, abort_after_s: 2.0}
  tol:   {y: 0.012, th_deg: 20, tgt_y: 0.004, tgt_th_deg: 1.5}
```

`Qy`, `Ru` and `rho_s` are derived values; recompute them from the tolerances and increments in a startup check and refuse to start if the YAML disagrees.

10. Calibration and bench procedures, in order

a. IMU (one afternoon). 1. Confirm axes: tilt the robot nose-down, `a_x` of the raw accelerometer must go positive; rotate counter-clockwise seen from above, gyro `z` positive. 2. Stationary noise: 60 s log at 200 Hz after a 5 min warm-up. `sigma_a` and `sigma_r` are the sample standard deviations of linear acceleration and gyro `z`. Expect 0.01 to 0.03 m/s^2 and 0.1 to 0.2 deg/s for the BNO085. 3. Zeroing residual: zero as in Section 5, then push the robot 1 m and back by hand, stop, and read the mean linear acceleration over 2 s. That residual is the `bias_a` the design treats as noise; if it exceeds 0.05 m/s^2, the attitude estimate of the BNO085 is drifting with motion and `q_v` must go up (re-run the NEES check in simulation with the measured value). 4. Gyro bias after zeroing: should be below 0.1 deg/s.

b. Camera and fiducial (one day). 1. Intrinsics with a chequerboard (OpenCV), 30+ views, reprojection error below 0.3 px. 2. Static noise versus range: robot stationary at 0.3, 0.6, 1.0 and 1.5 m, straight on and at 15 degrees yaw, 300 frames each. Record the standard deviation of `x`, `y`, `theta` of `z` (after the chain). These fill `sigma_p0`, `d_0`, `sigma_th`. Expect 2 to 5 mm and 0.3 to 1 degree at 1 m with a 100 mm tag. 3. Latency: flash an LED in the camera's

view and log the time between the GPIO edge and the frame stamp; confirm the stamp is at exposure. Record the processing delay distribution; it is not used by the filter but must be stable. 4. Sign conventions: the four known poses of Section 2. All four must agree with the tape measure to within the noise before the EKF is switched on. 5. Marker offset `(m, psi_0)` and the coupler offset `L_k`: measured from the physical target and robot, written into the YAML.

c. Wheels (half a day). 1. Wheel radius under load, `l_x`, `l_y`: tape measure. 2. IK signs: command each wheel alone through the wheel node and confirm the direction; then command pure forward, pure left and pure yaw and confirm the chassis motion matches `test_mpc_kinematics`. 3. Speed step response: from rest, step one wheel to 5 rad/s and log the measured speed at 200 Hz; fit a first-order response; that is `tau`. Repeat for a 10 rad/s step and for all four wheels. If `tau` is above 0.2 s, plan on the lag model (design doc 4.8). 4. Maximum wheel speed under load: `w_max`. 5. Slip: drive straight at 0.2 m/s for 2 m, measure the actual distance; the residual sets the slip offset the simulation assumed (0.01 m/s).

d. Timing (an hour). With everything running and the robot on blocks, record `solve_ms` for 5 minutes and the inter-arrival times of `/imu/data` and `/dock/estimate`. Budgets of Section 4 must hold at the 99th percentile.

---

11. Test plan, in order

Mirror the MATLAB tests; do not invent new criteria.

Replay tests (desk, no robot). Export the MATLAB sensor fixtures to CSV (a `export_fixtures.m` script: IMU samples, fiducial fixes with capture and availability stamps, truth) and replay them through the Python EKF. The Python output must match `matlab/reference/ekf/run_ekf` to 1e-9 on the same inputs, since both are the same arithmetic. Likewise dump one MPC tick from MATLAB (`x_hat`, `u_prev`, `y_r`, `lim`, and the resulting `dU`) and check the Python tick against it to the solver tolerance. These two files are the acceptance test for the port.

Unit tests (`pytest`). Jacobians against finite differences; prediction matrices against brute-force stepping; constraint rows against the limits for a tick where the cap binds; wrap-around of the heading innovation; ring-buffer lookup at, before and between stamps.

Bench, robot on blocks. Wheels free. Feed the real IMU and camera; the MPC must command sensible velocities when the marker is moved by hand. Check `solve_ms`, `fallback` count, and that the wheel node's saturation never engages.

Hardware, open loop. Robot pushed by hand with the marker visible: the EKF estimate against the tape measure at rest, and the NIS histogram over a session (mean near 3). Then the delay test: with the robot rolling at a steady 0.2 m/s, the estimate must not lag the fixes (the delay figure of the simulation, now on hardware).

Hardware, closed loop, staged. (1) APPROACH only, `v_approach = 0.10`, stop at the standoff. (2) Full sequence at `v_final = 0.02`. (3) Nominal caps. (4) Thirty runs from the three start poses of `test_mpc_closed_loop` and random ones, with the final pose measured externally (a second camera looking down at a marker on the robot, or a dial gauge at the coupler). The scatter of final lateral and yaw error against the 12 mm and 20 degree box is the main result; docking time, penetration and contact speed distributions beside it. These are the hardware counterparts of `report_demo_montecarlo.png`.

Record every run as a bag. The thesis figures come from bags, not from screen captures.

---

12. Safety rules for the closed-loop runs

- Hardware E-stop within reach, tested at the start of every session.
- The wheel node zeros on command timeout (100 ms), on E-stop, and on `command_enabled = false`.
- Caps in the YAML for the first runs: `v_approach = 0.10`, `v_final = 0.02`, `w_max` halved. Raise only after ten clean runs.
- The supervisor aborts on covariance blow-up (`p_diag[0] > 0.05^2`), on the corridor being violated by the estimate, and on two consecutive MPC fallbacks.
- A physical stop behind the target so a runaway cannot leave the table.

---

13. Map from the MATLAB to the port

MATLAB	Python	Equation sheet	Test

`ekf_predict.m`	`dock_control/ekf.py: Ekf.predict`	`ekf_equations.pdf` 3 to 6	replay, finite-difference Jacobians
`ekf_update.m`	`Ekf.update`	7 to 9	replay, wrap test
`run_ekf.m` (schedule)	`ekf_node.py`	10	replay
fiducial pre-transform (not in MATLAB)	`fiducial_node.py: chain()`	7, Section 2 here	four-pose sign test
`mecanum_ik.m`, `mecanum_fk.m`	`kinematics.py`	design doc Appendix A	wheel sign test
`mpc_linearise.m`	`mpc.py: linearise`	bridge Step 1	finite differences
`mpc_predict_matrices.m`	`mpc.py: predict_matrices`	Step 3	brute force
`mpc_constraints.m`	`mpc.py: constraints`	Step 6	cap binds, row count
`mpc_step.m`	`mpc.py: Mpc.step` (OSQP)	Steps 4 to 7	tick dump, closed loop
`run_closed_loop.m` supervisor	`supervisor_node.py`	design doc 4.7	staged hardware runs
`dock_params.m`	`config/dock_demo.yaml`	design doc Section 7	startup consistency check

14. Deliverables for the thesis from this work

- Calibration tables: IMU noise and residual bias, fiducial noise versus range, `tau` per wheel, `w_max`, geometry.
- The replay-test result (Python equals MATLAB) as the evidence that the port is the design.
- Timing: `solve_ms` and loop-period histograms on the Pi.
- The hardware NIS histogram and the EKF-versus-tape-measure table.
- Thirty closed-loop runs: final-pose scatter, docking time, penetration and contact speed, success rate, all from bags.
- One annotated run: raw fixes, estimate, commands, stage, with the simulation's equivalent figure beside it.